Below is the Case Study for your University Scholarship Allocation System, written in the same format as the Campus Festival example.

Case Study: University Scholarship Allocation System

Problem Statement

Design a system that evaluates students for scholarships using aggregate marks, subject-wise minimum scores, attendance, and family income. The system ranks eligible students and resolves ties using predefined rules.

Objective

Check scholarship eligibility based on marks, subjects, attendance, and income.

Rank eligible students.

Resolve ties using attendance and family income.

Display the final scholarship list.

Assumptions

Student data is already stored in a list.

Minimum aggregate marks = 75.

Subject marks must be at least 60.

Minimum attendance = 75%.

Family income must not exceed ■50,000.

Ranking priority:

Higher aggregate marks

Higher attendance

Lower family income

Algorithm

Start.

Store student details in a list.

Create an empty list called eligible.

Check each student's marks, subject scores, attendance, and income.

If all conditions are satisfied, add the student to eligible.

Sort eligible students by marks, attendance, and lower income.

Display ranks and student details.

Display the total number of eligible students.

Stop.

Flowchart

[START]

|

[Store Student Data]

|

[Check Eligibility]

|

Eligible?

/ \

No Yes

| |

[Ignore] [Add to List]

\ /

[Repeat]

|

[Sort by Marks, Attendance,

Lower Income]

|

[Display Ranking]

|

[Display Total Eligible]

|

[STOP]

Pseudocode

START

Create eligible list

FOR each student

IF marks ≥ 75 AND

subject1 ≥ 60 AND

subject2 ≥ 60 AND

attendance ≥ 75 AND

income ≤ 50000

Add student to eligible

ENDIF

ENDFOR

Sort eligible by marks, attendance, and lower income

Display ranking

Display total eligible students

STOP

Python Source Code

```
students = [
["Ravi", 90, 85, 92, 95, 40000],
["Sita", 90, 80, 88, 90, 30000],
["Arun", 75, 70, 65, 80, 20000],
["Priya", 85, 90, 95, 95, 25000],
]
```

```
eligible = []
for s in students:
name, marks, sub1, sub2, attendance, income = s
if (marks >= 75 and sub1 >= 60 and sub2 >= 60
```

and attendance >= 75 and income <= 50000):

eligible.append(s)

eligible.sort(key=lambda s: (-s[1], -s[4], s[5]))

```
print("SCHOLARSHIP ELIGIBLE STUDENTS")
print("-----------------------------")
rank = 1
for s in eligible:
print(rank, s[0], "- Marks:", s[1],
```

"Attendance:", s[4], "%")

rank += 1

```
print("\nTotal eligible students:", len(eligible))
```

Test Case

Input

Student data:

Name	Marks	Sub1	Sub2	Attendance	Income
Ravi	90	85	92	95	40000
Sita	90	80	88	90	30000
Arun	75	70	65	80	20000
Priya	85	90	95	95	25000

Output

SCHOLARSHIP ELIGIBLE STUDENTS

---

1 Ravi - Marks: 90 Attendance: 95 %

2 Sita - Marks: 90 Attendance: 90 %

3 Priya - Marks: 85 Attendance: 95 %

4 Arun - Marks: 75 Attendance: 80 %

Total eligible students: 4

Conclusion

This program uses Python lists, loops, conditional statements, and sorting to identify eligible students, rank them fairly, and display the final scholarship list based on predefined eligibility rules.

Case Study: Multi-Event Sports Tournament Scheduling and Ranking System

Problem Statement

Multi-Event Sports Tournament Scheduling and Ranking System: Manage team scores using Python dictionaries, calculate points based on wins, compare tie-break scores, arrange teams according to their performance, and display the final tournament rankings.

Objective

Store team names and their scores using dictionaries.

Accept team names, wins, and tie-break scores.

Calculate points based on the number of wins.

Update team scores in the dictionary.

Rank teams based on total points.

Use tie-break scores when teams have equal points.

Display the final tournament rankings.

Assumptions

Team details are stored using a dictionary.

Each team has points and a tie-break score.

Each win gives 3 points.

Team names are entered by the user.

Higher points receive a better rank.

If points are equal, the higher tie-break score gets a better rank.

Three team entries are accepted from the user.

Algorithm

Start.

Create a dictionary called teams with team names, points, and tie-break scores.

Repeat three times:

Read the team name.

Read the number of wins.

Read the tie-break score.

Calculate points as wins × 3.

Store the team details in the dictionary.

Sort the teams based on:

Higher points first.

Higher tie-break score next.

Display the tournament rankings.

Display each team's rank, name, points, and tie-break score.

Stop.

Flowchart

[START]

|

[Create Teams Dictionary]

|

[Repeat 3 Times]

|

[Enter Team Name]

|

[Enter Wins]

|

[Enter Tie-break Score]

|

[Calculate Points]

```
Points = Wins * 3
```

|

[Update Team Details]

|

[More Teams?]

/ \

Yes No

| |

[Repeat] [Sort Teams]

|

[Points Highest First]

|

[Compare Tie-break Score]

|

[Display Rankings]

|

[STOP]

Pseudocode

```
START
teams = dictionary with team details
FOR i = 1 TO 3
```

Read team name

Read wins

Read tie-break score

```
points = wins * 3
```

Store team details in dictionary

ENDFOR

Sort teams by:

Higher points

Higher tie-break score

Display tournament rankings

FOR each team

Display rank, team name, points,

and tie-break score

ENDFOR

```
STOP
```

Python Source Code

```
# Sports Tournament Ranking Systemteams = {"Team A": [3, 10], "Team B": [3, 15], "Team C": [1, 8]}for i in range(3): name = input("Enter team name: ") win = int(input("Enter wins: ")) score = int(input("Enter tie-break score: ")) teams[name] = [win * 3, score]ranking = sorted(teams.items(), key=lambda x: (-x[1][0], -x[1][1]))print("Tournament Rankings:")for rank, (name, values) in enumerate(ranking, 1): print(rank, name, "Points:", values[0], "Tie-break:", values[1])
```

Test Cases

Test Case 1: Teams have different points

Input:

Enter team name: Team D

Enter wins: 5

Enter tie-break score: 20

Enter team name: Team E

Enter wins: 4

Enter tie-break score: 15

Enter team name: Team F

Enter wins: 2

Enter tie-break score: 10

Output:

Tournament Rankings:

1 Team D Points: 15 Tie-break: 20

2 Team E Points: 12 Tie-break: 15

3 Team A Points: 3 Tie-break: 10

4 Team B Points: 3 Tie-break: 15

5 Team C Points: 1 Tie-break: 8

6 Team F Points: 6 Tie-break: 10

```
Note: The correct sorted order for the above data is:
```

Tournament Rankings:

1 Team D Points: 15 Tie-break: 20

2 Team E Points: 12 Tie-break: 15

3 Team F Points: 6 Tie-break: 10

4 Team B Points: 3 Tie-break: 15

5 Team A Points: 3 Tie-break: 10

6 Team C Points: 1 Tie-break: 8

Test Case 2: Two teams have equal points

Input:

Enter team name: Team D

Enter wins: 3

Enter tie-break score: 20

Enter team name: Team E

Enter wins: 3

Enter tie-break score: 10

Enter team name: Team F

Enter wins: 2

Enter tie-break score: 15

Output:

Tournament Rankings:

1 Team D Points: 9 Tie-break: 20

2 Team E Points: 9 Tie-break: 10

3 Team F Points: 6 Tie-break: 15

4 Team B Points: 3 Tie-break: 15

5 Team A Points: 3 Tie-break: 10

6 Team C Points: 1 Tie-break: 8

Conclusion

This program manages sports tournament rankings using Python dictionaries, loops, sorting, and conditional operations. It calculates team points based on wins, compares tie-break scores, and displays the final rankings. It helps organize tournament results and identify the winning teams.

Case Study: Campus Festival Registration and Financial Audit

Problem Statement

Campus Festival Registration and Financial Audit: Manage participant registrations by detecting duplicate names, calculate total registration fees and sponsorship income, compare the total income with festival expenses, and determine whether the festival results in a profit or loss.

Objective

Accept the number of participants and their names.

Detect duplicate registrations.

Calculate total registration fees.

Accept sponsorship and total expenses.

Calculate the final financial balance.

Identify whether the festival makes a profit or loss.

Assumptions

Participant names are entered by the user.

Duplicate names are not registered again.

Registration fees are collected only from valid participants.

Sponsorship and expenses are entered in rupees.

A zero or positive balance indicates profit; a negative balance indicates loss.

Algorithm

Start.

Create an empty list called participants.

```
Initialize fees = 0.
```

Read the number of participants.

```
Repeat for each participant:
```

Read the participant's name.

Check whether the name already exists in the list.

If it exists, display "Duplicate registration!".

Otherwise, add the name to the list.

Read the registration fee and add it to total fees.

Read the sponsorship amount.

Read the total expenses.

Calculate the balance using:

```
balance = fees + sponsorship - expenses
```

Display the number of valid participants and financial report.

If balance is greater than or equal to zero, display "Profit"; otherwise, display "Loss".

Stop.

Flowchart

[START]

|

[Create Empty List

```
  and Set Fees = 0 ]
```

|

[Enter Number of

Participants]

|

[Enter Participant Name]

|

[Name Already Exists?]

/ \

Yes No

| |

[Duplicate Message] [Add Name]

|

[Enter Registration Fee]

|

[Update Total Fees]

|

[More Participants?]

/ \

Yes No

| |

[Repeat] [Enter Sponsorship

and Expenses]

|

[Calculate Balance]

|

[Display Report]

|

[Balance >= 0?]

/ \

Yes No

| |

[Profit] [Loss]

| |

+----+-----+

|

[STOP]

Pseudocode

```
START
participants = empty list
fees = 0
```

Read n

FOR each participant

Read name

IF name exists in participants

Display "Duplicate registration!"

ELSE

Add name to participants

Read fee

```
  fees = fees + fee
```

ENDIF

ENDFOR

Read sponsorship

Read expenses

```
  balance = fees + sponsorship - expenses
```

Display number of valid participants

Display fees, sponsorship, expenses, balance

IF balance >= 0

Display "Festival Result: Profit"

ELSE

Display "Festival Result: Loss"

ENDIF

```
  STOP
```

Python Source Code

```
  # Campus Festival Registration and Financial Auditparticipants = []fees = 0n = int(input("Enter number of
  participants: "))for i in range(n): name = input("Enter participant name: ") if name in participants:
  print("Duplicate registration!") else: participants.append(name) fee = int(input("Enter registration fee:
  ")) fees = fees + feesponsorship = int(input("Enter sponsorship amount: "))expenses = int(input("Enter total
  expenses: "))balance = fees + sponsorship - expensesprint("\n--- Festival Report ---")print("Valid
  Participants:", len(participants))print("Registration Fees:", fees)print("Sponsorship:",
  sponsorship)print("Expenses:", expenses)print("Final Balance:", balance)if balance >= 0: print("Festival
  Result: Profit")else: print("Festival Result: Loss")
```

Test Cases

Test Case 1: Festival makes a profit

Input:

Enter number of participants: 3

Enter participant name: Ravi

Enter registration fee: 500

Enter participant name: Sita

Enter registration fee: 500

Enter participant name: Kumar

Enter registration fee: 500

Enter sponsorship amount: 2000

Enter total expenses: 2500

Output:

--- Festival Report ---

Valid Participants: 3

Registration Fees: 1500

Sponsorship: 2000

Expenses: 2500

Final Balance: 1000

Festival Result: Profit

Test Case 2: Duplicate registration

Input:

Enter number of participants: 3

Enter participant name: Ravi

Enter registration fee: 500

Enter participant name: Sita

Enter registration fee: 500

Enter participant name: Ravi

Duplicate registration!

Enter sponsorship amount: 500

Enter total expenses: 2000

Output:

Duplicate registration!

--- Festival Report ---

Valid Participants: 2

Registration Fees: 1000

Sponsorship: 500

Expenses: 2000

Final Balance: -500

Festival Result: Loss

Conclusion

This program manages campus festival registrations and finances using Python lists, loops, conditional statements, and arithmetic operations. It prevents duplicate registrations, calculates total income and expenses, and determines whether the festival makes a profit or incurs a loss.

Top of Form

Bottom of Form

Case Study: Bus Passenger Management and Capacity Monitoring

Problem Statement

Bus Passenger Management and Capacity Monitoring: Manage passenger boarding and alighting at multiple bus stops, calculate the current number of passengers, detect invalid passenger counts, and identify whether the bus exceeds its seating capacity.

Objective

Accept the bus capacity.

Accept the number of bus stops.

Read the number of passengers boarding and alighting at each stop.

Calculate the current passenger count.

Detect invalid passenger counts.

Identify capacity violations.

Display the passenger count at each stop.

Assumptions

Bus capacity is entered as an integer.

The initial number of passengers is zero.

Passengers boarding and alighting are entered by the user.

Passenger counts cannot be negative.

The number of passengers should not exceed bus capacity.

Each stop is processed one by one.

Algorithm

Start.

Read the bus capacity.

```
Initialize passengers = 0.
```

Read the number of stops.

```
Repeat for each stop:
```

Read the number of passengers boarding.

Read the number of passengers alighting.

```
Calculate the current passengers:
passengers = passengers - exit_bus + board
```

If passengers are less than zero, display "Invalid passenger count!".

If passengers exceed capacity, display "Capacity violation!".

Otherwise, display the stop number and passenger count.

Repeat until all stops are completed.

Stop.

Flowchart

[START]

|

[Enter Bus Capacity]

|

[Passengers = 0]

|

[Enter Number

of Stops]

|

[Enter Boarding

Passengers]

|

[Enter Alighting

Passengers]

|

[Update Passenger Count]

|

[Passengers < 0?]

/ \

Yes No

| |

[Invalid Count] [Passengers > Capacity?]

/ \

Yes No

| |

[Capacity [Display Stop

Violation] and Passengers]

| |

+-----+----+

|

[More Stops?]

/ \

Yes No

| |

[Repeat] [STOP]

Pseudocode

START

Read capacity

passengers = 0

Read stops

FOR each stop

Read board

Read exit_bus

```
  passengers = passengers - exit_bus + board
```

IF passengers < 0

Display "Invalid passenger count!"

ELSE IF passengers > capacity

Display "Capacity violation!"

ELSE

Display stop number and passengers

ENDIF

ENDFOR

```
  STOP
```

Python Source Code

```
# Bus Passenger Managementcapacity = int(input("Enter bus capacity: "))passengers = 0stops =
int(input("Enter number of stops: "))for i in range(stops): board = int(input("Passengers boarding: "))
exit_bus = int(input("Passengers alighting: ")) passengers = passengers - exit_bus + board if passengers <
0: print("Invalid passenger count!") elif passengers > capacity: print("Capacity violation!") else:
print("Stop", i+1, "Passengers:", passengers)
```

Test Cases

Test Case 1: Normal passenger count

Input:

Enter bus capacity: 50

Enter number of stops: 2

Passengers boarding: 20

Passengers alighting: 0

Passengers boarding: 10

Passengers alighting: 5

Output:

Stop 1 Passengers: 20

Stop 2 Passengers: 25

Test Case 2: Capacity violation

Input:

Enter bus capacity: 30

Enter number of stops: 1

Passengers boarding: 40

Passengers alighting: 0

Output:

Capacity violation!

Test Case 3: Invalid passenger count

Input:

Enter bus capacity: 50

Enter number of stops: 1

Passengers boarding: 5

Passengers alighting: 10

Output:

Invalid passenger count!

Conclusion

This program manages bus passengers using Python loops, conditional statements, and arithmetic operations. It calculates passenger counts at each stop, detects invalid passenger counts, and identifies capacity violations to ensure safe bus transportation.

Top of Form

Think

Bottom of Form

Case Study: Campus Examination Seat Allocation and Validation System

Problem Statement

Campus Examination Seat Allocation and Validation System: Manage student seat allocations using nested lists, identify reserved seats, detect empty seats and invalid allocations, and find duplicate roll numbers to ensure proper examination seating arrangements.

Objective

Store student roll numbers using nested lists.

Identify reserved seat positions.

Detect empty seats.

Identify invalid seat allocations.

Detect duplicate student roll numbers.

Display valid seat allocations.

Assumptions

Student roll numbers are stored in a nested list.

A positive number represents a valid student roll number.

Zero represents an empty seat.

A negative number represents an invalid allocation.

Reserved seat positions are stored as tuples.

Duplicate roll numbers are not considered valid allocations.

Seat positions are represented using row and column indexes.

Algorithm

Start.

Create a nested list called seats.

Create a list of reserved seat positions.

Extract all positive roll numbers into a list called rolls.

```
Repeat for each row in the seating arrangement:
Repeat for each seat in the row.
```

Check whether the seat position is reserved.

If reserved, display "Reserved position".

If the roll number is zero, display "Empty seat".

If the roll number is negative, display "Invalid allocation".

If the roll number occurs more than once, display "Duplicate roll number".

Otherwise, display "Valid seat".

Repeat until all seats are checked.

Stop.

Flowchart

[START]

|

[Create Seats List

and Reserved List]

|

[Extract Positive

Roll Numbers]

|

[Read Row and Column]

|

[Check Seat Position]

|

[Reserved?]

/ \

Yes No

| |

[Reserved [Roll == 0?]

Position] / \

Yes No

| |

[Empty Seat] [Roll < 0?]

/ \

Yes No

| |

[Invalid] [Duplicate?]

[Allocation] / \

Yes No

| |

[Duplicate] [Valid Seat]

| |

+-----+----+

|

[More Seats?]

/ \

Yes No

| |

[Repeat] [STOP]

Pseudocode

```
START
seats = nested list
reserved = list of reserved positions
rolls = all positive roll numbers from seats
```

FOR each row in seats

FOR each seat in row

IF position is reserved

Display "Reserved position"

ELSE IF roll == 0

Display "Empty seat"

ELSE IF roll < 0

Display "Invalid allocation"

ELSE IF roll occurs more than once

Display "Duplicate roll number"

ELSE

Display "Valid seat"

ENDIF

ENDFOR

ENDFOR

```
STOP
```

Python Source Code

```
# Campus Examination Seat Allocationseats = [[101, 102, 0], [103, 104, 105], [106, 101, -1]]reserved = [(0, 2), (2, 2)]rolls = [x for row in seats for x in row if x > 0]for i, row in enumerate(seats): for j, roll in enumerate(row): if (i, j) in reserved: print(roll, "Reserved position") elif roll == 0: print("Empty seat at", i, j) elif roll < 0: print("Invalid allocation at", i, j) elif rolls.count(roll) > 1: print("Duplicate roll number:", roll) else: print("Valid seat:", roll)
```

Test Cases

Test Case 1: Normal and valid seat allocation

Input:

```
seats = [[101, 102, 103], [104, 105, 106]]
reserved = [(0, 2)]
```

Output:

Valid seat: 101

Valid seat: 102

103 Reserved position

Valid seat: 104

Valid seat: 105

Valid seat: 106

Test Case 2: Duplicate, empty and invalid seats

Input:

```
seats = [[101, 102, 0], [103, 101, -1]]
reserved = []
```

Output:

Duplicate roll number: 101

Valid seat: 102

Empty seat at 0 2

Valid seat: 103

Duplicate roll number: 101

Invalid allocation at 1 2

Test Case 3: Reserved seat allocation

Input:

```
seats = [[201, 202, 203]]
reserved = [(0, 1)]
```

Output:

Valid seat: 201

202 Reserved position

Valid seat: 203

Conclusion

This program manages examination seat allocations using Python nested lists, tuples, loops, conditional statements, and list comprehension. It identifies reserved seats, empty seats, invalid allocations, and duplicate roll numbers to ensure proper examination seating management.

Do you like this personality?

Case Study: Inventory Management and Order Reconciliation System

Problem Statement

Inventory Management and Order Reconciliation System: Manage product orders by checking ordered, dispatched, and returned quantities. Validate inventory records, identify inconsistent order details, calculate outstanding orders, and update the current inventory based on returned products.

Objective

Accept the ordered quantity.

Accept the dispatched quantity.

Accept the returned quantity.

Accept the current inventory.

Detect invalid records.

Identify inconsistent order details.

Calculate outstanding orders.

Update the inventory after returns.

Assumptions

All quantities are entered as integers.

Negative quantities are not allowed.

Dispatched quantity should not exceed ordered quantity.

Returned quantity should not exceed dispatched quantity.

Current inventory cannot be negative.

Outstanding orders are calculated as ordered quantity minus dispatched quantity.

Updated inventory is calculated by adding returned quantity to current inventory.

Algorithm

Start.

Read the ordered quantity.

Read the dispatched quantity.

Read the returned quantity.

Read the current inventory.

Check whether any quantity is negative.

If yes, display "Invalid record".

Otherwise, check whether dispatched quantity is greater than ordered quantity or returned quantity is greater than dispatched quantity.

If yes, display "Inconsistent records".

Otherwise:

Calculate outstanding orders as ordered - dispatched.

Calculate updated inventory as stock + returned.

Display outstanding orders.

Display updated inventory.

Display "Valid order records".

Stop.

Flowchart

[START]

|

[Enter Ordered Quantity]

|

[Enter Dispatched Quantity]

|

[Enter Returned Quantity]

|

[Enter Current Inventory]

|

[Any Quantity Negative?]

/ \

Yes No

| |

[Invalid [Dispatched > Ordered

Record] OR Returned > Dispatched?]

/ \

Yes No

| |

[Inconsistent [Calculate

Records] Outstanding Orders]

|

[Update Inventory]

|

[Display Results]

|

[Valid Records]

|

[STOP]

Pseudocode

START

Read ordered

Read dispatched

Read returned

Read stock

IF ordered < 0 OR dispatched < 0

OR returned < 0 OR stock < 0

Display "Invalid record"

ELSE IF dispatched > ordered

OR returned > dispatched

Display "Inconsistent records"

ELSE

```
outstanding = ordered - dispatched
updated_stock = stock + returned
```

Display outstanding orders

Display updated inventory

Display "Valid order records"

ENDIF

```
STOP
```

Python Source Code

```
# Inventory Management Systemordered = int(input("Enter ordered quantity: "))dispatched = int(input("Enter
dispatched quantity: "))returned = int(input("Enter returned quantity: "))stock = int(input("Enter current
inventory: "))if ordered < 0 or dispatched < 0 or returned < 0 or stock < 0: print("Invalid record")elif
dispatched > ordered or returned > dispatched: print("Inconsistent records")else: print("Outstanding
orders:", ordered - dispatched) print("Updated inventory:", stock + returned) print("Valid order records")
```

Test Cases

Test Case 1: Valid order records

Input:

Enter ordered quantity: 100

Enter dispatched quantity: 70

Enter returned quantity: 10

Enter current inventory: 50

Output:

Outstanding orders: 30

Updated inventory: 60

Valid order records

Test Case 2: Invalid record

Input:

Enter ordered quantity: -100

Enter dispatched quantity: 70

Enter returned quantity: 10

Enter current inventory: 50

Output:

Invalid record

Test Case 3: Inconsistent records

Input:

Enter ordered quantity: 100

Enter dispatched quantity: 120

Enter returned quantity: 10

Enter current inventory: 50

Output:

Inconsistent records

Test Case 4: Returned quantity exceeds dispatched quantity

Input:

Enter ordered quantity: 100

Enter dispatched quantity: 70

Enter returned quantity: 80

Enter current inventory: 50

Output:

Inconsistent records

Conclusion

This program manages product orders and inventory using Python variables, conditional statements, and arithmetic operations. It detects invalid and inconsistent records, calculates outstanding orders, and updates inventory based on returned products.

Top of Form

Bottom of Form

Case Study: Room Electricity and Water Consumption Audit

Problem Statement

Room Electricity and Water Consumption Audit: Manage room-wise electricity and water usage, calculate the total charges for each room, identify rooms with high consumption, and display a warning when electricity or water usage exceeds the specified limits.

Objective

Store room numbers and their electricity and water usage.

Calculate electricity charges at ■5 per unit.

Calculate water charges at ■2 per unit.

```
Calculate the total charge for each room.
```

Detect high electricity or water consumption.

Display the room-wise usage and consumption status.

Assumptions

Room numbers are stored as dictionary keys.

```
Electricity and water usage are stored in a list for each room.
```

Electricity costs ■5 per unit.

Water costs ■2 per unit.

Electricity usage above 150 units or water usage above 60 units is considered high consumption.

Otherwise, consumption is considered normal.

Algorithm

Start.

Create a dictionary containing room numbers and their electricity and water usage.

```
Repeat for each room in the dictionary.
```

Read the room number and its usage values.

Calculate the electricity and water charges.

Calculate the total charge.

Display the room number and total charge.

If electricity usage is above 150 or water usage is above 60, display "Warning: High consumption!"

Otherwise, display "Normal consumption".

Display electricity and water usage.

Stop.

Flowchart

```
START
```

Create room dictionary with usage values

Select each room and read usage

Calculate electricity charge + water charge

Electricity above 150 OR water above 60?

Yes

Display high-consumption warning

No

Display normal consumption

Display room charges and usage

```
STOP
```

Pseudocode

```
START
rooms = dictionary of room numbers and usage
```

FOR each room and usage in rooms

```
electricity, water = usage
charge = electricity * 5 + water * 2
```

Display room number and charge

IF electricity > 150 OR water > 60

Display "Warning: High consumption!"

ELSE

Display "Normal consumption"

ENDIF

Display electricity and water usage

ENDFOR

```
STOP
```

Python Source Code

```
# Room Electricity and Water Consumption Audit
rooms = {"101": [120, 50], "102": [200, 80], "201": [90, 40]}
for room, usage in rooms.items():
electricity, water = usage
charge = electricity * 5 + water * 2
print("Room", room, "Charge: Rs.", charge)
if electricity > 150 or water > 60:
print("Warning: High consumption!")
else:
print("Normal consumption")
print("Electricity:", electricity, "Water:", water)
```

Test Cases

Test Case 1: Normal consumption

Input:

```
Room 101: Electricity = 120, Water = 50
```

Output:

Room 101 Charge: Rs. 700

Normal consumption

Electricity: 120 Water: 50

Calculation:

Test Case 2: High consumption

Input:

```
Room 102: Electricity = 200, Water = 80
```

Output:

Room 102 Charge: Rs. 1160

Warning: High consumption!

Electricity: 200 Water: 80

Calculation:

Test Case 3: Normal consumption

Input:

```
Room 201: Electricity = 90, Water = 40
```

Output:

Room 201 Charge: Rs. 530

Normal consumption

Electricity: 90 Water: 40

Calculation:

Conclusion

This Python program manages room-wise electricity and water consumption using dictionaries, loops, conditional statements, and arithmetic operations. It calculates charges for each room and identifies high consumption by displaying appropriate warnings.

Top of Form

Bottom of Form

Case Study: Environmental Monitoring Data Analyzer

Problem Statement

Environmental Monitoring Data Analyzer: Analyze air-quality or water-quality readings using nested lists, calculate the average, highest, and lowest readings, and classify each observation as High or Normal based on a given threshold.

Objective

Store environmental readings using nested lists.

Calculate the average of all readings.

Identify the highest reading.

Identify the lowest reading.

Classify readings as High or Normal.

Display the environmental monitoring report.

Assumptions

Environmental readings are entered as numerical values.

Data is stored using nested lists.

A reading greater than 75 is classified as High.

A reading less than or equal to 75 is classified as Normal.

The dataset contains at least one reading.

Algorithm

Start.

Create a nested list called data containing environmental readings.

Create an empty list called readings.

Use a nested loop to collect all readings into a single list.

Calculate the average using the sum of readings divided by the total number of readings.

Find the highest reading using max().

Find the lowest reading using min().

Display the average, highest, and lowest readings.

Use nested loops to check each reading.

If a reading is greater than 75, display "High"; otherwise, display "Normal".

Stop.

Flowchart

[START]

|

[Create Nested List]

|

[Collect All Readings]

|

[Calculate Average]

|

[Find Highest Reading]

|

[Find Lowest Reading]

|

[Display Results]

|

[Check Each Reading]

|

[Reading > 75?]

/ \

Yes No

| |

[Display [Display

High] Normal]

| |

+-----+-----+

|

[More Readings?]

/ \

Yes No

| |

[Repeat] [STOP]

Pseudocode

```
START
data = nested list of readings
readings = empty list
```

FOR each row in data

FOR each value in row

Add value to readings

ENDFOR

ENDFOR

```
avg = sum(readings) / length(readings)
highest = maximum(readings)
lowest = minimum(readings)
```

Display average

Display highest

Display lowest

FOR each row in data

FOR each value in row

IF value > 75

Display "High"

ELSE

Display "Normal"

ENDIF

ENDFOR

ENDFOR

```
STOP
```

Python Source Code

```
# Environmental Monitoring Data Analyzerdata = [[45, 60, 70], [80, 90, 55], [30, 40, 65]]readings = [x for row in data for x in row]avg = sum(readings) / len(readings)print("Average reading:", round(avg, 2))print("Highest reading:", max(readings))print("Lowest reading:", min(readings))for i, row in enumerate(data): for value in row: if value > 75: print("Row", i+1, value, "- High") else: print("Row", i+1, value, "- Normal")
```

Test Cases

Test Case 1: Environmental readings analysis

Input:

```
data = [[45, 60, 70], [80, 90, 55], [30, 40, 65]]
```

Output:

Average reading: 59.44

Highest reading: 90

Lowest reading: 30

Row 1 45 - Normal

Row 1 60 - Normal

Row 1 70 - Normal

Row 2 80 - High

Row 2 90 - High

Row 2 55 - Normal

Row 3 30 - Normal

Row 3 40 - Normal

Row 3 65 - Normal

Test Case 2: All readings are normal

Input:

```
data = [[20, 30, 40], [50, 60, 70]]
```

Output:

Average reading: 45.0

Highest reading: 70

Lowest reading: 20

Row 1 20 - Normal

Row 1 30 - Normal

Row 1 40 - Normal

Row 2 50 - Normal

Row 2 60 - Normal

Row 2 70 - Normal

Conclusion

This program analyzes environmental monitoring data using Python nested lists, loops, and conditional statements. It calculates the average, highest, and lowest readings and classifies each observation as High or Normal. It helps monitor environmental conditions and identify readings that exceed the specified threshold.